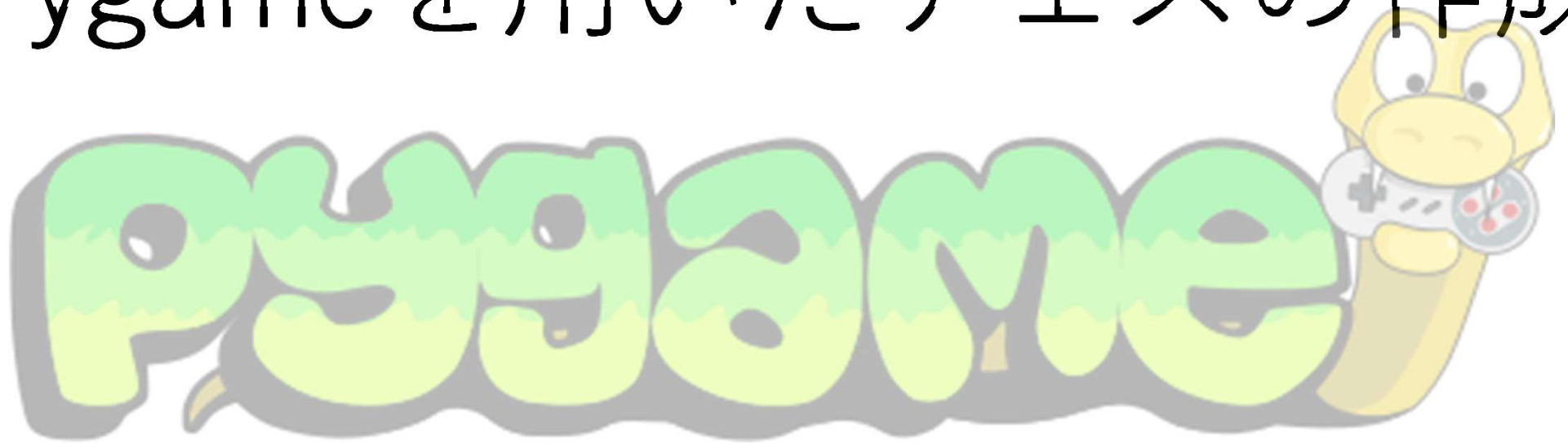


Pygameを用いたチェスの作成



概要

Pygameとは...

- ・ Pythonでゲームを作成するためのライブラリ
- ・ 2Dゲームを簡単に作れるように、画像の描画、キーボードやマウスの入力処理などの機能を提供



- ・ ゲーム用のウィンドウを表示
 - ・ 各駒に対応する画像の表示
- ・ ボードの操作をカーソルでおこなう

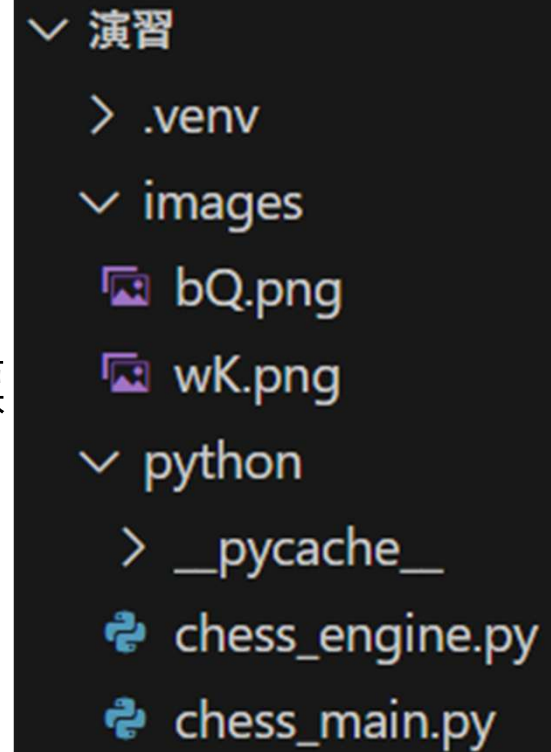
入出力

入力：

- ・ 各駒に対応する画像ファイル（imagesフォルダ）
- ・ マウスがクリックされた際のウィンドウ上の座標

出力：

- ・ Pygameを用いたウィンドウの表示
- ・ 駒の移動が正常におこなわれたかの表示



処理

1. Pygameの初期化&画面作成
2. 画像のロード（駒の読み込み）
3. ウィンドウの表示
4. マウスによる入力を取得
5. 4の入力を基に駒の移動が可能か確認
6. 移動不可能なら3に戻る
移動可能なら駒を移動 & ターンの切り替え（白⇔黒）
7. チェックメイトか確認
8. チェックメイトなら処理終了
チェックメイトでなければ3に戻る

コード (一部抜粋)

```
BOARD_WIDTH = BOARD_HEIGHT = 480
DIMENSION = 8
SQUARE_SIZE = BOARD_HEIGHT // DIMENSION
IMAGES = {}
COLORS = ["#FFF5E1", "#855E42"]
```

```
def load_images():
    pieces = ["bR", "bN", "bB", "bQ", "bK", "bP", "wP", "wK", "wQ", "wB", "wN", "wR"]
    for piece in pieces:
        IMAGES[piece] = pygame.transform.scale(pygame.image.load("images/" + piece + ".png"), (SQUARE_SIZE, SQUARE_SIZE))
```

```
def draw_game_status(screen, gs):
    draw_board(screen)
    draw_pieces(screen, gs.board)
```

```
def draw_board(screen):
    for row in range(DIMENSION):
        for col in range(DIMENSION):
            color = COLORS[((row+col)%2)]
            pygame.draw.rect(screen, color, pygame.Rect(col*SQUARE_SIZE, row*SQUARE_SIZE, SQUARE_SIZE, SQUARE_SIZE))
```

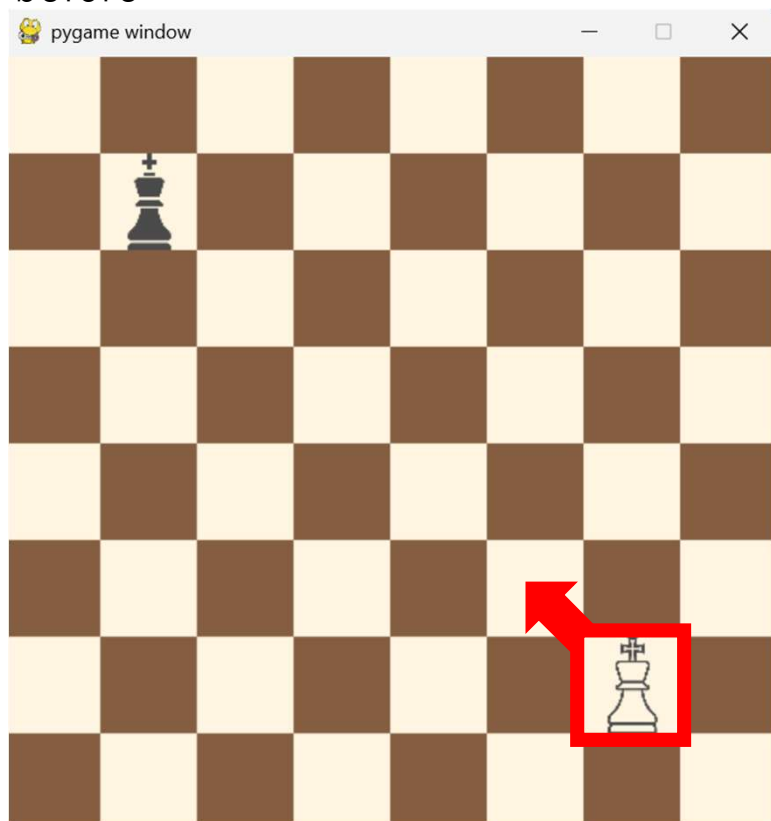
```
def draw_pieces(screen, board):
    for row in range(DIMENSION):
        for col in range(DIMENSION):
            piece = board[row][col]
            if piece != " ":
                screen.blit(IMAGES[piece], pygame.Rect(col*SQUARE_SIZE, row*SQUARE_SIZE, SQUARE_SIZE, SQUARE_SIZE))
```

```
def get_King_moves(self, player_click):
    origin_row, origin_col = player_click[0]
    dest_row, dest_col = player_click[1]
    if(self.board[origin_row][origin_col][0]=="w" and self.player):
        if(abs(origin_row-dest_row) <= 1 and abs(origin_col-dest_col) <= 1):
            return True
        else:
            return False
    elif(self.board[origin_row][origin_col][0] == "b" and not self.player):
        if(abs(origin_row-dest_row) <= 1 and abs(origin_col-dest_col) <= 1):
            return True
        else:
            return False
    else:
        return False
```

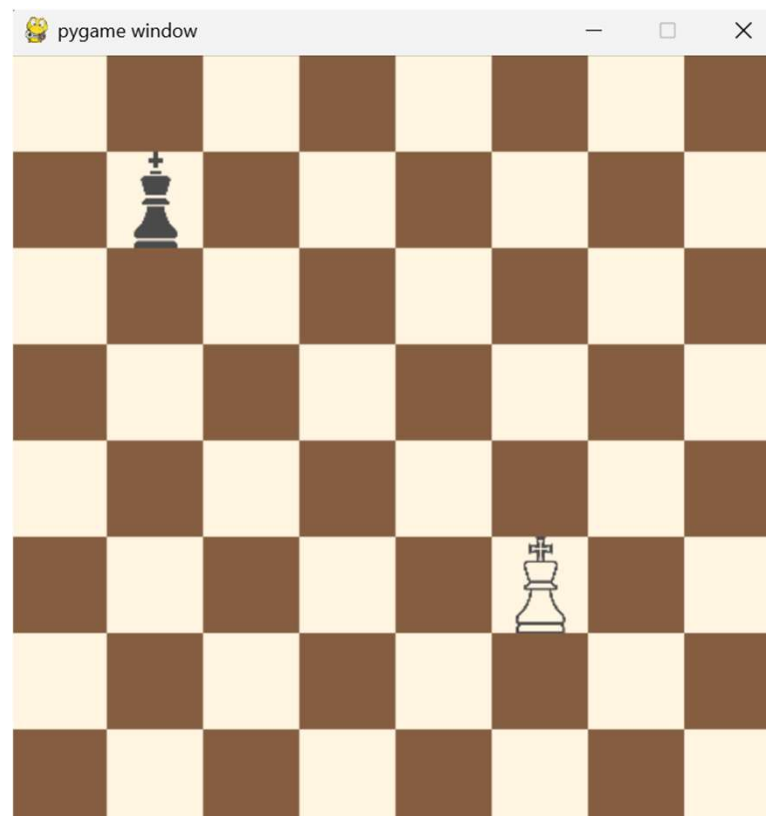
コード (キングの処理)

出力結果

before



after



ターミナル>>>

Success!
Black turn: